Evaluating Agent Harness Components for Language Models

Проблема

Модели допускают ошибки, забывают о них и неэффективно используют инсайты об ошибках, так как пытаются улучшить сами себя.

Одна и та же LLM не может гарантировать улучшение, если она сама допустила ошибку.

Что говорит литература

1

Без внешнего сигнала модель не исправляет себя

Самокоррекция GPT-3.5 на CommonSenseQA: 75,8% → 41,8%. Рефлексия помогает сильным моделям, у слабой результат не меняется.

Huang et al., ICLR 2024; Reflexion, NeurIPS 2023

2

Внешний детерминированный сигнал помогает

Без линтера, отклоняющего правку с ошибкой, доля решённых задач падает с 18,0% до 15,0%.

CRITIC, ICLR 2024; SWE-agent, NeurIPS 2024

3

Эффект харнесса зависит от размера модели

Больше всего выигрывают средние модели, слабые не используют добавленные артефакты. Модели меньше 32B не проверялись.

Lin et al., EMNLP 2026

4

Проверять скриптами и повторами

Тесты вместо LLM-судьи, изолированное окружение, несколько прогонов: у GPT-4o 61% по pass^1, но меньше 25% по pass^8.

SkillsBench, 2026; AppWorld, ACL 2024; τ-bench, ICLR 2025

Подход

- Модели делю по размеру: малые помещаются в ~20 ГБ на компьютере. Мобильные не тестирую: в большинстве бенчмарков упрёмся в их способности.
- Одинаковое подготовленное окружение (sandbox) и один harness.
- Система отслеживает подключённые инструменты, обобщённо плагины.

Модели: семейство Qwen3.5

Малая	Qwen3.5-27B, Qwen3.5-35B-A3B
Средняя	Qwen3.5-122B-A10B
Большая	облачная Qwen

Что тестирую

- **Умное сжатие контекста.**
- **Статичное улучшение на допущенных ошибках** (задачи с кодом): ошибка записывается в определённом формате как инструкция и добавляется в контекст при следующем прогоне.

Принципы подготовки датасетов

Подготовлю свои принципы по образцу SkillsBench (2026).

Задачи

- Настоящая работа, а не учебные, выдуманные, игрушечные или синтетические задачи.
- Проверка детерминированными pass/fail-тестами, без LLM-as-a-judge.
- Сложность из-за самой задачи, а не из-за запутанной инструкции.
- Решаема от начала до конца без заранее заложенных ответов и «магических» чисел.
- Инструкции пишет человек; если LLM помогала с формулировкой, финальную версию правит человек.

Прогон

- Изолированное окружение.
- Задачу решает агент в харнессе.
- Результат проверяет скрипт.

Датасеты и бенчмарки

Набор	Задачи	Как проверяется	Статус
PMo / BPMN руководитель, MODELS 2026	55 сценариев: текст процесса → BPMN-диаграмма в Modelio	Исполнение скрипта, структура против эталона	Первый датасет
SkillsBench 2026	87 задач в 8 областях	Детерминированные pass/fail-тесты, изолированное окружение	Кандидат
AppWorld ACL 2024	750 задач, 9 приложений, 457 API	Unit-тесты по финальному состоянию	Кандидат
SWE-bench ICLR 2024	Исправление реальных issue в репозиториях	Тесты репозитория	Кандидат, задачи с кодом

Метрики, которые собираю

Качество

Доля решённых задач по статичным правилам (основная метрика). pass^k: решена ли задача во всех k независимых прогонах.

Стоимость

Время выполнения задачи, потраченные токены, число вызовов модели, число повторов.

Работа инструментов

Какие инструменты и плагины использовались; успешные и неудачные вызовы инструментов и хуков; размер результата инструмента.

Артефакты и трейс

Все созданные артефакты и полный трейс действий агента в харнессе: по ним можно найти уникальные ошибки и понять, где компонент помог или навредил.

Оценка эффекта и вклад работы

Проверка

- Тот же sandbox, те же задачи и модель; меняется только компонент.
- Несколько независимых прогонов на каждую задачу и метрика pass^k из τ-bench [11]: задача засчитывается, только если решена во всех k прогонах. У GPT-4o 61% по pass^1, но меньше 25% по pass^8.
- Сравниваю метрики с компонентом и без него для каждого размера модели.

Новизна

- Влияние компонентов харнесса на малых моделях: в SWE-agent компоненты отключали по одному только на GPT-4 Turbo, у Lin et al. минимум 32B.
- Исправление ошибок вынесено в детерминированную часть харнесса.
- Все артефакты и трейсы открыты: видно, что изменилось и чего это стоило.

Литература

1. J. Huang et al. Large Language Models Cannot Self-Correct Reasoning Yet. ICLR, 2024. arxiv.org/abs/2310.01798
2. N. Shinn et al. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS, 2023. arxiv.org/abs/2303.11366
3. Z. Gou et al. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. ICLR, 2024. arxiv.org/abs/2305.11738
4. J. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS, 2024. arxiv.org/abs/2405.15793
5. C. S. Xia et al. Agentless: Demystifying LLM-based Software Engineering Agents. FSE, 2025. arxiv.org/abs/2407.01489
6. M. Lin et al. Harness Updating Is Not Harness Benefit: Disentangling Evolution Capabilities in Self-Evolving LLM Agents. EMNLP, 2026. aclanthology.org/2026.emnlp-main.2326
7. Better Harnesses, Smaller Models: Building 90% Cheaper Agents via Automated Harness Adaptation. Препринт, 2026. arxiv.org/abs/2607.08938

Литература

8. HarnessX: A Composable, Adaptive, and Evolvable Agent Harness Foundry. Препринт, 2026. arxiv.org/abs/2606.14249
9. X. Li et al. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. NeurIPS, 2026. arxiv.org/abs/2602.12670
10. H. Trivedi et al. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. ACL, 2024. aclanthology.org/2024.acl-long.850
11. S. Yao et al. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. ICLR, 2025. arxiv.org/abs/2406.12045
12. X. Liu et al. AgentBench: Evaluating LLMs as Agents. ICLR, 2024. arxiv.org/abs/2308.03688
13. C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. arxiv.org/abs/2310.06770
14. Towards LLM-Assisted Business Process Modeling in an Industrial Modeling Tool: An Experience Report. MODELS, 2026. hal.science/hal-05702678v1